MOMO Quant

Part 10 — Final Documentation Package & Cursor Handoff Guide

Version: 1.0 Draft
Primary Goal: Prepare the complete documentation package so Cursor can generate the project in a controlled, module-by-module way.

---

1. Purpose

This document explains how to hand MOMO Quant documentation to Cursor and how to use Cursor without letting it corrupt the architecture.

The goal is not to ask Cursor to “build the bot.”

The goal is to make Cursor follow a controlled engineering plan.

Cursor should generate code only after reading the relevant documentation and only for one milestone at a time.

---

2. Required Documentation Package

The `/docs` folder must contain these files:

```
/docs/01-srs.md
/docs/02-system-architecture.md
/docs/03-database-design.md
/docs/04-api-specification.md
/docs/05-ai-ml-design.md
/docs/06-ui-ux-specification.md
/docs/07-devops-deployment.md
/docs/08-cursor-implementation-plan.md
/docs/09-testing-qa-plan.md
/docs/10-cursor-handoff-guide.md
```

These documents are the official source of truth.

Cursor must be instructed not to ignore them.

---

3. Documentation Reading Order

Cursor should read the documents in this order:

```
1. 01-srs.md
2. 02-system-architecture.md
3. 03-database-design.md
4. 04-api-specification.md
5. 05-ai-ml-design.md
6. 06-ui-ux-specification.md
7. 07-devops-deployment.md
8. 08-cursor-implementation-plan.md
9. 09-testing-qa-plan.md
10. 10-cursor-handoff-guide.md
```

For each milestone, Cursor should also be told which documents matter most.

Example:

```
For database work:
- 02-system-architecture.md
- 03-database-design.md
- 08-cursor-implementation-plan.md
- 09-testing-qa-plan.md
```

Example:

```
For AI service work:
- 05-ai-ml-design.md
- 04-api-specification.md
- 08-cursor-implementation-plan.md
- 09-testing-qa-plan.md
```

---

4. Cursor Master Instruction

Use this as the first instruction when opening the project in Cursor:

```
You are working on MOMO Quant, an AI-assisted quantitative crypto futures
trading platform.
```

```
Before writing code, read the /docs folder.

The documents are the source of truth.

Follow these rules:

1. Build the system module by module.
2. Do not generate the whole application at once.
3. Use .NET 8 for the backend.
4. Use React + Vite + Tailwind CSS for the dashboard.
5. Use MySQL with Entity Framework Core.
6. Use Redis for runtime cache/state where needed.
7. Use Python FastAPI for the AI service.
8. Build as a modular monolith first.
9. Do not create microservices except the Python AI service.
10. Do not implement real live trading in MVP.
11. Do not allow AI to place orders.
12. Do not allow strategies to place orders.
13. Risk engine must approve every trade before execution.
14. Backtesting, replay, paper trading, and future live trading must share
common abstractions.
15. Every important trading decision must be logged.
16. All timestamps must use UTC.
17. All financial trading values must use decimal, not float or double.
18. Use standard API response format from the API specification.
19. Use role-based authorization.
20. Dangerous actions must create audit logs.
21. Live trading must remain disabled until readiness checks pass.
22. Return full updated files when modifying code.
23. Add tests for important logic.
24. Do not invent architecture that conflicts with the documentation.
```

---

5. Cursor Milestone Workflow

Each milestone should follow this process:

```
1. Start a new Git branch.
2. Give Cursor the milestone prompt from 08-cursor-implementation-plan.md.
3. Tell Cursor which docs to read.
4. Let Cursor create or modify files.
5. Build the project.
6. Run tests.
7. Manually inspect generated code.
```

```
8. Fix issues.
9. Commit only when the milestone works.
10. Move to the next milestone.
```

Do not run multiple milestones together.

That is how architecture gets messy.

---

6. Correct Cursor Prompt Format

Every Cursor prompt should follow this structure:

```
Context:
You are working on MOMO Quant. Read the relevant docs in /docs.

Relevant docs:
- [list docs]

Task:
Implement [specific module].

Requirements:
[list exact requirements]

Architecture rules:
[list safety/architecture rules]

Acceptance criteria:
[list what must work]

Output:
Return full updated files.
```

---

7. Example Cursor Prompt — Milestone 1

```
Context:
You are working on MOMO Quant, an AI-assisted quantitative crypto futures
trading platform.

Relevant docs:
```

```
- /docs/01-srs.md
- /docs/02-system-architecture.md
- /docs/08-cursor-implementation-plan.md
- /docs/09-testing-qa-plan.md
- /docs/10-cursor-handoff-guide.md

Task:
Implement Milestone 1 — Repository and Solution Skeleton.

Requirements:
Create the repository structure for:

- .NET 8 backend
- React + Vite frontend
- Python FastAPI AI service
- Docker deployment files
- scripts folder
- docs folder

Inside src/backend, create a .NET 8 solution named MomoQuant.sln with these
projects:

- MomoQuant.Api
- MomoQuant.Application
- MomoQuant.Domain
- MomoQuant.Infrastructure
- MomoQuant.Persistence
- MomoQuant.Worker
- MomoQuant.Shared
- MomoQuant.UnitTests
- MomoQuant.IntegrationTests

Architecture rules:
- Do not implement trading logic yet.
- Do not implement live trading.
- Follow the folder structure from the docs.
- Set correct project references.
- Keep the solution clean and buildable.

Acceptance criteria:
- Solution builds.
- All projects exist.
- Project references are correct.
- No trading logic is implemented yet.
- Repository structure matches the documentation.

Output:
Return full created/updated files.
```

8. Example Cursor Prompt — Milestone 3

```
Context:
You are working on MOMO Quant.

Relevant docs:
- /docs/02-system-architecture.md
- /docs/03-database-design.md
- /docs/08-cursor-implementation-plan.md
- /docs/09-testing-qa-plan.md
- /docs/10-cursor-handoff-guide.md

Task:
Implement Milestone 3 — Database Entities and EF Core Setup.

Requirements:
Create the persistence layer using Entity Framework Core and MySQL.

Use Pomelo.EntityFrameworkCore.MySql.

Create MomoQuantDbContext.

Map the MVP database tables from the database design document.

Configure:
- decimal precision decimal(28,12)
- UTC timestamp fields
- required indexes
- unique candle index
- relationships
- delete restrictions for trading records

Architecture rules:
- Domain logic must not depend on EF Core.
- Database logic must stay in Persistence.
- Trading records must not be casually deleted.
- Financial values must use decimal.
- Timestamps must be UTC.
- API keys must not be stored raw.

Acceptance criteria:
- Database project builds.
- Migration can be created.
- DbContext contains MVP tables.
- Indexes are configured.
```

```
- Decimal precision is configured.
- No plain-text API secrets are exposed.

Output:
Return full created/updated files.
```

---

9. Example Cursor Prompt — Milestone 8

```
Context:
You are working on MOMO Quant.

Relevant docs:
- /docs/01-srs.md
- /docs/02-system-architecture.md
- /docs/05-ai-ml-design.md
- /docs/08-cursor-implementation-plan.md
- /docs/09-testing-qa-plan.md
- /docs/10-cursor-handoff-guide.md

Task:
Implement Milestone 8 — Strategy Framework.

Requirements:
Create:

- ITradingStrategy
- StrategyContext
- StrategySignalResult
- StrategyEngine
- Strategy registry
- Strategy parameter provider

Implement MVP strategies:

1. LiquiditySweepStrategy
2. VwapMeanReversionStrategy
3. EmaPullbackStrategy

Architecture rules:
- Strategies only produce signals.
- Strategies must not place orders.
- Strategies must not approve risk.
- Strategies must return human-readable reasons.
- Strategies must declare supported regimes and timeframes.
```

```
- Indicator calculations must not be inside strategies.

Acceptance criteria:
- Strategy engine runs enabled strategies.
- Each strategy returns signal/no-trade result.
- Signals are stored.
- Strategies do not execute trades.
- Strategy parameters can be changed.
- Unit tests cover basic signal behavior.

Output:
Return full updated files.
```

---

10. Example Cursor Prompt — Milestone 12

```
Context:
You are working on MOMO Quant.

Relevant docs:
- /docs/01-srs.md
- /docs/02-system-architecture.md
- /docs/03-database-design.md
- /docs/04-api-specification.md
- /docs/05-ai-ml-design.md
- /docs/08-cursor-implementation-plan.md
- /docs/09-testing-qa-plan.md
- /docs/10-cursor-handoff-guide.md

Task:
Implement Milestone 12 — Backtesting Engine.

Requirements:
Create:

- IBacktestEngine
- BacktestRunner
- BacktestExecutionProvider
- BacktestPortfolio
- BacktestPerformanceCalculator
- BacktestController

Backtest flow:
- load candles
- calculate/load indicators
```

```
- detect market regime
- run strategy engine
- call AI confidence engine
- run risk engine
- simulate execution
- store signals, AI decisions, risk decisions, orders, fills, trades
- calculate results

Execution simulation must include:
- maker fee
- taker fee
- slippage assumption
- order timeout
- missed maker orders

Architecture rules:
- Backtesting must reuse the same strategy, AI, risk, and execution abstractions
as paper/live.
- Do not fake profitability by assuming every limit order fills.
- Store rejected signals and missed orders.
- Store every major decision.
- Use decimal values for financial calculations.

Acceptance criteria:
- Backtest can run from API.
- Backtest stores decisions and trades.
- Backtest results are calculated.
- Missed maker orders are logged.
- Results are realistic enough for MVP.
- Tests cover key backtest flow.

Output:
Return full updated files.
```

---

11. What Cursor Must Not Do

Cursor must not:

```
Build the entire app in one prompt.
Invent a new architecture.
Implement live trading early.
Let AI place orders.
Let strategies place orders.
Skip the risk engine.
```

```
Skip audit logs.
Use float/double for money.
Ignore UTC timestamps.
Hide trading decisions.
Hardcode exchange keys.
Expose secrets.
Use frontend logic for trading decisions.
Create unnecessary microservices.
Add Kubernetes early.
Add complex AI before rule-based MVP works.
```

---

12. Required Human Review

Cursor-generated code must be reviewed manually.

Focus review on:

```
Architecture boundaries
Risk engine logic
Execution simulation realism
Database relationships
Decimal precision
Authorization
Secret handling
Audit logging
Error handling
Backtest assumptions
Paper trading behavior
```

Do not trust generated trading code blindly.

Cursor can help write code, but you are responsible for the system.

---

13. First Implementation Sequence

The first real build sequence should be:

```
1. Save all docs in /docs.
2. Create Git repository.
3. Create feature/milestone-01-skeleton.
```

```
4. Run Cursor Milestone 1 prompt.
5. Build solution.
6. Commit.
7. Create feature/milestone-02-domain.
8. Run Cursor Milestone 2 prompt.
9. Build and test.
10. Commit.
11. Continue milestone by milestone.
```

---

14. Local Setup Before Cursor Builds Features

Before serious implementation, make sure local machine has:

```
.NET 8 SDK
Node.js LTS
Python 3.11+
Docker Desktop
MySQL client optional
Redis client optional
Git
Cursor
```

Recommended local services:

```
MySQL in Docker
Redis in Docker
.NET API local
React dashboard local
Python AI local
```

---

15. Documentation Update Rule

If implementation reveals a better design, update the docs.

Do not let code and docs drift apart.

Process:

```
1. Identify required design change.
2. Update relevant document.
3. Continue implementation.
4. Mention the change in commit message.
```

The docs are not frozen forever, but changes must be intentional.

---

16. Handoff Checklist

Before giving docs to Cursor, confirm:

```
All 10 docs exist.
File names are correct.
Docs are in /docs.
Cursor master instruction is ready.
Milestone prompts are ready.
Git repository is initialized.
Development environment is installed.
Docker Desktop works.
No real secrets are present.
Live trading remains out of MVP.
```

---

17. Final Handoff Summary

Cursor should be treated like a junior developer with high coding speed.

It needs:

```
Clear architecture
Clear boundaries
Clear rules
Clear acceptance criteria
Small tasks
Manual review
Tests
```

The most important handoff rule:

```
Never ask Cursor to build MOMO Quant.
Ask Cursor to build the next verified milestone.
```